

Lab 4 — Transformers and Dyck Languages

Error Detection, Correction, and Interpretability

Anthony Saab

GitHub repository: <https://github.com/anthonyysaab/lab4-dyck-transformer>

Overview

This report studies whether a small Transformer encoder can learn the structure of Dyck languages. The model is trained on bracket strings from $D(2)$ using the bracket types $()$ and $[]$. The main tasks are binary error detection, local correction, out-of-distribution generalisation, attention analysis, and probing.

The main result is negative but informative. The Transformer performs well in-distribution, but it does not learn a robust stack-like rule. It reaches high accuracy on depths seen during training, but fails on deeper OOD strings. Fine-tuning and curriculum learning improve OOD accuracy, but neither gives reliable symbolic generalisation.

1 Chomsky hierarchy, Dyck languages, and structural generalisation

Dyck languages are **context-free languages**. A Dyck language $D(k)$ contains well-balanced bracket strings over k bracket types. In this lab, $k = 2$:

$$D(2) = \{\text{well-balanced strings over } (), []\}.$$

A regular language can be recognised by a finite-state automaton. That is not enough for full Dyck recognition, because unbounded Dyck recognition requires remembering an unbounded number of open brackets. For example:

([([])])

The correct closer depends on the most recent unmatched opener. This requires a stack: push an expected closer when an opener appears, compare each closer with the top of the stack, and pop if it matches.

Therefore $D(k)$ is context-free, not regular, for $k \geq 1$. It can be recognised by a pushdown automaton, but not by a finite-state automaton when nesting depth is unbounded.

A stronger theoretical result is the Chomsky–Schützenberger theorem: every context-free language can be represented as the homomorphic image of the intersection of a Dyck language with a regular language. Intuitively, the Dyck language supplies the nested stack structure, while the regular language filters the allowed derivation patterns. This is why $D(2)$ is often treated as a universal skeleton for context-free structure: it captures the balanced, hierarchical part of context-free computation, and a homomorphism maps that abstract bracket structure back to the terminal symbols of a particular language.

This makes Dyck languages useful for testing structural generalisation. A model that learns the true rule should generalise to longer strings and deeper nesting. A model that only learns training-distribution cues should fail outside the training depth range.

In this experiment, the training, development, and in-distribution test splits use depths up to 4. The OOD split uses depths 5–7 and longer strings. This directly tests whether the Transformer learned a stack-like algorithm.

2 Real-world examples reducible to $D(k)$

Programming-language delimiters

Programming languages use nested parentheses, brackets, and braces:

```
function_call(array[index])
```

Each delimiter type can be treated as a bracket type:

```
( )  
[ ]  
{ }
```

This string is valid:

```
foo([x])
```

This string is invalid:

```
foo([x])
```

The second example is a type mismatch: the opener `[` expects `]`, but the string gives `)`. This corresponds to the `E3_type_mismatch` corruption in the experiment.

In programming-language delimiters, E1 corresponds to a missing closing delimiter, E2 to an extra opening delimiter, E3 to a wrong closing delimiter type, and E4 to a closing delimiter appearing before any matching opener is available.

XML and HTML tags

XML and HTML also contain nested open and close tags:

```
<div><span>text</span></div>
```

Treating each tag type as a bracket type gives a Dyck-like structure:

```
<div>    = opener type div  
</div>   = closer type div  
<span>   = opener type span  
</span>  = closer type span
```

This string is invalid:

```
<div><span>text</div></span>
```

The closing tag `</div>` appears while the current stack top is `<span>`. This is again a type mismatch.

In XML or HTML, E1 corresponds to a missing closing tag, E2 to an extra opening tag, E3 to a mismatched closing tag, and E4 to a closing tag that appears before any compatible open tag.

3 LSTM memory, Transformer memory, and OOD nesting depth

For $D(1)$, an LSTM can approximate bounded-depth recognition with a counter-like hidden variable. With one bracket type, the model mostly needs the current depth:

$$\text{depth} = \#\text{openers} - \#\text{closers}.$$

For bounded depth n , it must distinguish:

$$0, 1, 2, \dots, n.$$

For $D(k)$ with $k \geq 2$, this is harder. The model must remember both depth and bracket type order. For $D(2)$, the stack could contain:

([([

The next valid closer must match the top of that stack. The number of possible stack states up to depth n grows roughly as:

$$1 + k + k^2 + \dots + k^n.$$

A fixed-width Transformer can compare token positions through self-attention, but it does not automatically implement an unbounded stack. It may learn patterns that work for depths seen during training without learning the general algorithm.

The OOD split is therefore the central test. If the model learned a stack-like rule, it should generalise from depths ≤ 4 to depths 5–7. If it learned distribution-specific cues, OOD performance should drop.

4 Dataset and model

The generated dataset is balanced between valid and invalid strings. Invalid strings are balanced across four corruption types:

- E1_missing_closer: one closing bracket is deleted;
- E2_spurious_opener: one extra opener is inserted;
- E3_type_mismatch: one closer has the wrong type;
- E4_premature_close: a closer appears where no opener is available.

Split	Rows	Valid	Invalid	Depth range
train	50000	25000	25000	1–4
dev	5000	2500	2500	1–4
test_id	5000	2500	2500	1–4
test_ood	5000	2500	2500	5–7

Table 1: Dataset splits. The OOD split contains deeper and longer strings than the training distribution.

The model is a small Transformer encoder with a fixed vocabulary:

[PAD], [CLS], [SEP], [UNK], (,), [,]

For detection, the final [CLS] representation is passed to a binary classifier. For correction, the model predicts one edit action per bracket-token position.

The main detector uses:

2 Transformer layers
4 attention heads
hidden size 64
feed-forward size 256
dropout 0.1
weight decay 1e-2
gradient clipping 1.0

5 Binary error detection

Overall results

The detector was trained for five epochs. The final test evaluation uses the checkpoint with the best development accuracy, not necessarily the final epoch.

Split	Loss	Accuracy	Macro-F1
dev, best checkpoint	0.3123	87.58%	87.40%
test_id	0.3113	87.64%	87.46%
test_ood	2.0376	38.74%	38.72%

Table 2: Binary detection results using the best development checkpoint.

The model performs well in-distribution, reaching 87.64% accuracy on `test_id`. But OOD accuracy drops to 38.74%, below chance for a balanced binary task. This is the first clear sign that the model did not learn a depth-general Dyck recogniser.

Confusion matrices

Gold / Predicted	Predicted invalid	Predicted valid
Gold invalid	1890	610
Gold valid	8	2492

Table 3: Confusion matrix on `test_id`.

In-distribution, the model almost always accepts valid strings. Its main error is falsely accepting invalid strings.

Gold / Predicted	Predicted invalid	Predicted valid
Gold invalid	928	1572
Gold valid	1491	1009

Table 4: Confusion matrix on `test_ood`.

On OOD data, both false acceptances and false rejections increase. The model no longer has a reliable decision boundary for deeper strings.

Error type

The most important qualitative result is that `E3_type_mismatch` is the hardest corruption. `E1`, `E2`, and `E4` often create simpler imbalance or local impossibility cues. `E3` is different: the sequence

may have the right number of openers and closers, but the closer has the wrong type. Detecting E3 requires comparing the closer with the most recent unmatched opener.

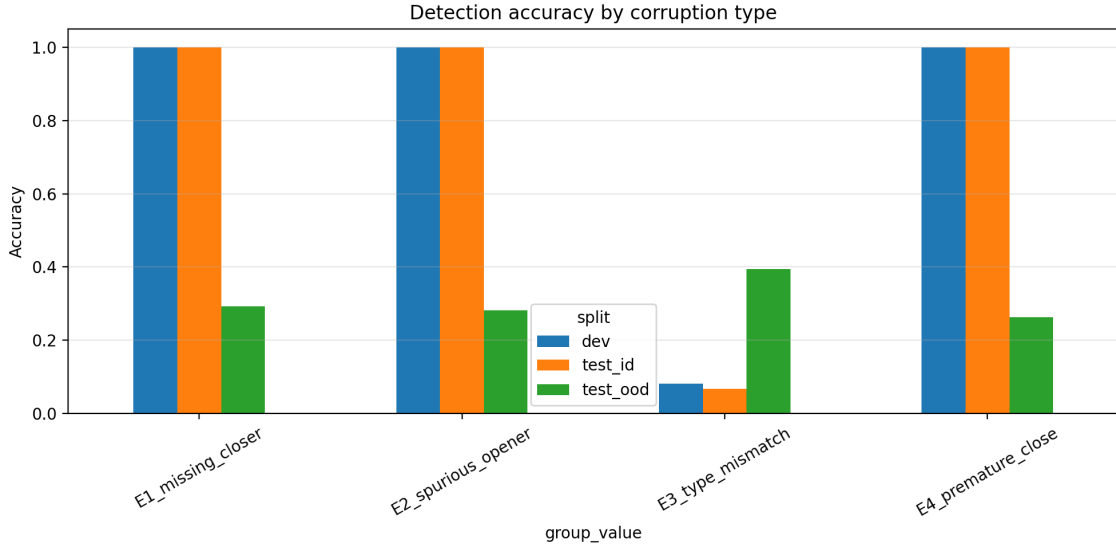


Figure 1: Detection accuracy by corruption type. Type mismatches are the most stack-sensitive error class.

The E3 weakness is important because it targets the core stack operation. The model learned useful cues, but not a reliable typed stack check.

6 Correction as sequence labelling

The correction task predicts one edit label per bracket token:

OK
DELETE
INSERT_LPAREN / INSERT_RPAREN / INSERT_LBRACK / INSERT_RBRACK
REPLACE_LPAREN / REPLACE_RPAREN / REPLACE_LBRACK / REPLACE_RBRACK

Special tokens and padding are ignored.

Correction results

Split	Token acc.	Macro-F1	Weighted-F1	Label exact	Repair valid	Invalid repair valid	Error-pos. acc.
test_id	98.13%	30.49%	97.42%	56.82%	56.82%	13.64%	14.28%
test_ood	99.20%	20.06%	98.84%	52.80%	52.80%	5.64%	5.54%

Table 5: Correction results. Repair valid means that the predicted edit sequence was applied to the input and the resulting string was accepted by the deterministic Dyck PDA. Invalid repair valid reports the same metric only on corrupted inputs.

Token accuracy is high but misleading. Most token labels are OK, so the model can score well without finding the actual corruption.

The repaired-string metric gives a clearer result. Overall repair validity is 56.82% on test_id and 52.80% on test_ood. However, this overall number includes valid examples, which the model

usually leaves unchanged. The more informative corrupted-only metric is much lower: only 13.64% of corrupted `test_id` strings and 5.64% of corrupted `test_ood` strings are successfully repaired into valid Dyck strings.

This confirms that the correction model mostly learns to preserve already-valid strings, but it rarely performs the exact local edit needed to repair corrupted ones. The low error-position accuracy supports the same conclusion: the model often fails to localise the actual corrupted token.

Limitations of the correction formulation

The correction labels are local, but some repairs require global stack information. Also, more than one edit can sometimes restore a valid Dyck string. The dataset has one gold repair, so label exact match penalises alternative valid repairs. Finally, label imbalance makes token accuracy over-optimistic.

7 Training and development curves

The detector converges quickly. Most improvement happens in the first epoch.

Epoch	Train loss	Train acc.	Dev loss	Dev acc.
1	0.4198	78.75%	0.3123	87.58%
2	0.3115	87.46%	0.3144	86.16%
3	0.3087	87.70%	0.3104	87.58%
4	0.3081	87.69%	0.3138	86.22%
5	0.3073	87.68%	0.3181	86.18%

Table 6: Training and development metrics over epochs.

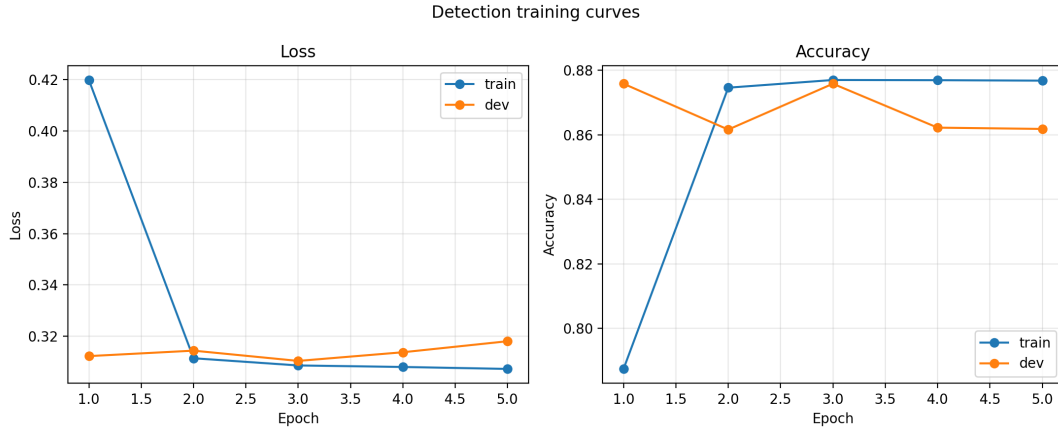


Figure 2: Training and development loss/accuracy over epochs.

The model reaches its best development accuracy early. After that, training loss continues to decrease slightly, but development accuracy does not improve. This points to mild overfitting and early saturation.

8 Out-of-distribution generalisation

The main OOD test asks whether a model trained on depths ≤ 4 generalises to depths 5–7.

Accuracy by nesting depth

Split	Max depth	Accuracy	Macro-F1
test_id	1–4	87.64%	87.46%
test_ood	5	39.57%	39.46%
test_ood	6	38.48%	38.47%
test_ood	7	38.18%	38.17%

Table 7: Base detector performance by depth. The OOD depth rows come from the depth-5 fine-tuning comparison table.

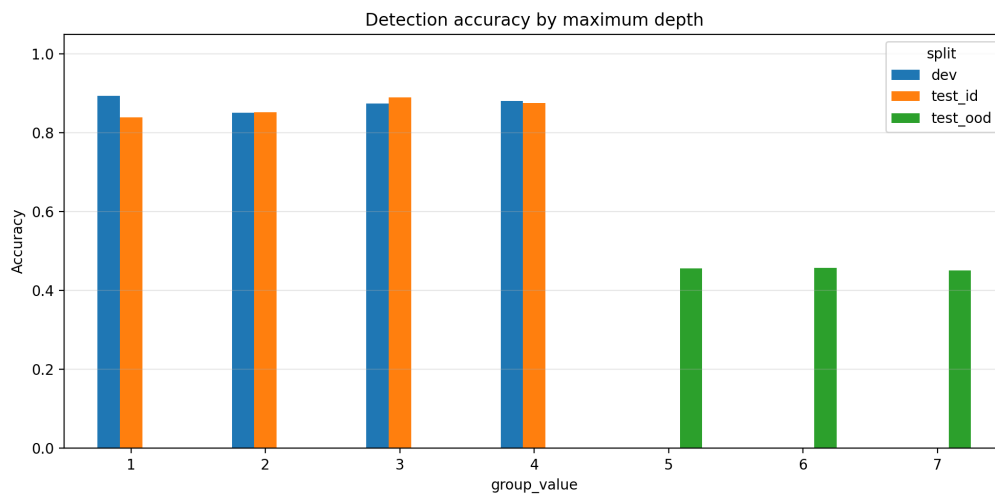


Figure 3: Detection accuracy by maximum nesting depth.

The model generalises to unseen examples from the same distribution, but not to deeper structures. This is not just memorisation of individual training strings, but it is also not abstract rule learning.

Accuracy by length

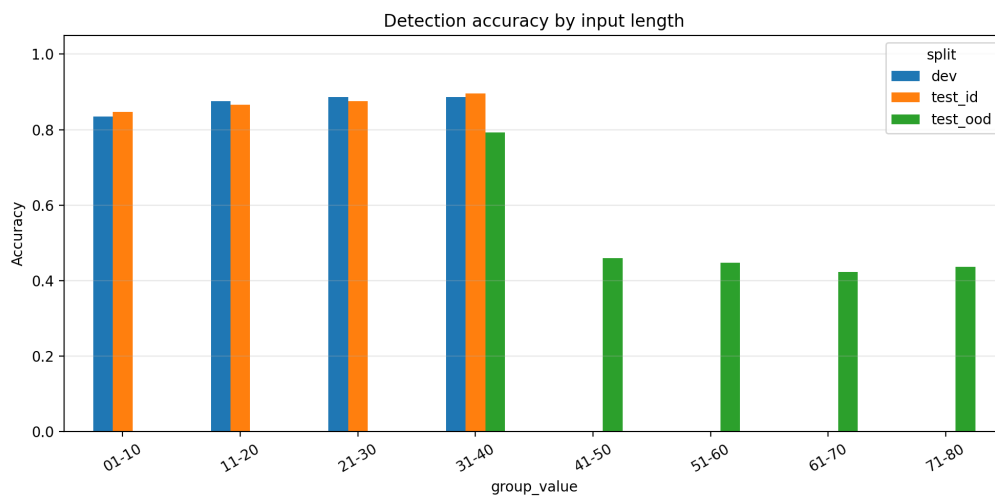


Figure 4: Detection accuracy by sequence length.

In-distribution, the model handles longer strings within the training depth range. The failure appears when length and depth both move outside the training distribution.

9 Comparison with a deterministic PDA baseline

The deterministic PDA baseline implements the exact Dyck algorithm:

1. scan the string left to right;
2. push expected closers for openers;
3. compare each closer with the stack top;
4. reject if the stack is empty, mismatched, or non-empty at the end;
5. accept otherwise.

Split	Rows	Accuracy	Macro-F1
train	50000	100%	100%
dev	5000	100%	100%
test_id	5000	100%	100%
test_ood	5000	100%	100%

Table 8: PDA baseline results.

System	test_id accuracy	test_ood accuracy
Transformer	87.64%	38.74%
PDA	100%	100%

Table 9: Transformer versus deterministic PDA.

The PDA is perfect because it implements the correct symbolic algorithm. The Transformer learns useful patterns, but not a reliable pushdown stack.

10 Fine-tuning on 500 depth-5 examples

The required fine-tuning experiment was implemented. The base detector was fine-tuned on 500 examples with exact depth 5, then evaluated on depths 5, 6, and 7.

Model	Depth	Rows	Accuracy	Macro-F1
Base detector	5	1668	39.57%	39.46%
Base detector	6	1666	38.48%	38.47%
Base detector	7	1666	38.18%	38.17%
Fine-tuned depth-5	5	1668	50.30%	34.00%
Fine-tuned depth-5	6	1666	50.30%	34.00%
Fine-tuned depth-5	7	1666	50.24%	33.86%

Table 10: Effect of fine-tuning on 500 depth-5 examples.

Fine-tuning raises raw OOD accuracy to about chance level. However, macro-F1 drops to about 34%, which suggests a class-biased solution rather than reliable Dyck-rule learning. The fine-tuned model does not show strong systematic extrapolation from depth 5 to depths 6 and 7.

11 OOD-improvement modification: curriculum learning

One OOD-improvement modification was implemented: curriculum training by maximum nesting depth. Instead of training on all depths ≤ 4 from the beginning, the model was trained in stages:

```
stage 1: max_depth <= 1
stage 2: max_depth <= 2
stage 3: max_depth <= 3
stage 4: max_depth <= 4
```

Split	Depth	Rows	Accuracy	Macro-F1
test_id	1	137	81.75%	81.07%
test_id	2	689	85.49%	85.40%
test_id	3	1026	89.38%	89.35%
test_id	4	3148	87.58%	87.20%
test_ood	5	1668	48.50%	46.34%
test_ood	6	1666	48.74%	47.31%
test_ood	7	1666	48.32%	46.89%

Table 11: Curriculum-trained detector results by depth.

Curriculum learning improves OOD performance relative to the base detector. The base model gets about 38–40% accuracy on depths 5–7, while the curriculum model gets about 48–49%.

This is an improvement, but it does not solve OOD generalisation. The model remains far below its in-distribution performance. Curriculum helps, but it does not make the Transformer behave like a PDA.

12 Attention heads and bracket matching

The attention analysis measures whether heads attend from closing brackets to their matching openers. For valid strings, matching bracket pairs are recovered with a stack, and attention mass is measured for each layer and head.

The assignment asks for average attention visualisations over correct Dyck strings. I inspected attention over valid strings, but focus the reported quantitative analysis on the more directly interpretable relation: attention from each closer to its stack-matched opener and from each opener to its matched closer. This is more stable than raw averaged matrices over variable-length bracket strings.

Matching attention on valid strings

Split	Layer/head	Pairs	Mean closer $\rightarrow$ opener	Std.	Median
test_id	layer 0, head 1	5982	0.0525	0.0390	0.0410
test_ood	layer 0, head 1	14576	0.0221	0.0125	0.0191

Table 12: Strongest matching-opener attention heads. Layer numbering starts at 0 in the implementation.

The strongest in-distribution head assigns only about 5.25% attention mass from a closer to its matching opener. On OOD strings, the strongest corresponding score is about 2.21%. The standard deviations are also reported, as required.

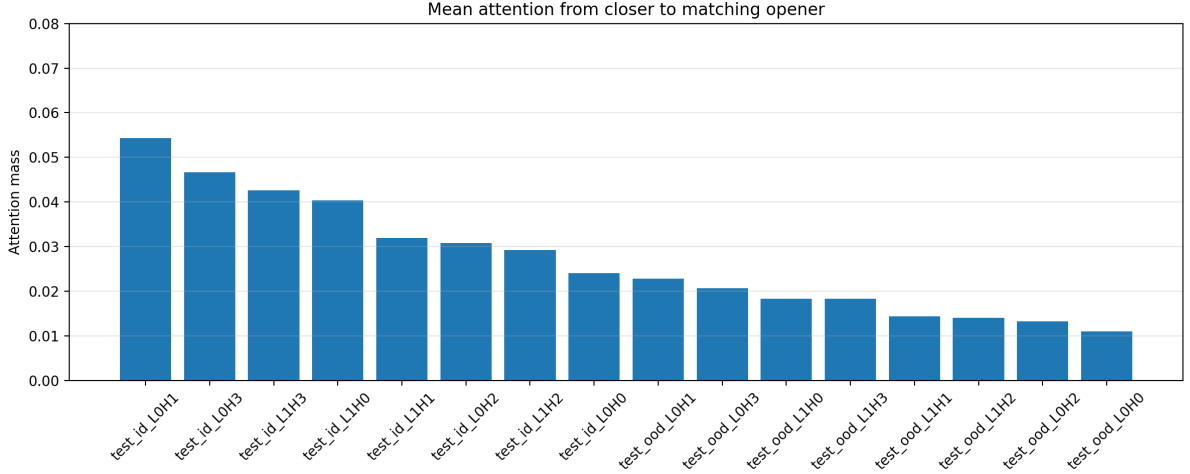


Figure 5: Mean attention mass from closing brackets to matching openers.

The result is weak but informative. Some heads are mildly sensitive to matching openers, but the attention pattern is not strong enough to count as a symbolic stack mechanism. The effect also weakens on OOD examples.

13 Attention on erroneous strings

A separate attention analysis was run on corrupted strings. Since some corrupted strings do not have well-defined matching pairs, this analysis focuses directly on the gold corrupted position.

For each erroneous example, the script measures attention involving the corrupted token:

- [CLS] → error token;
- error token → [CLS];
- error token → previous/next token;
- previous/next token → error token.

Split	Error type	Layer/head	Examples	Mean [CLS] → error	Std.
test_id	E1_missing_closer	layer 1, head 0	126	0.0461	0.0470
test_id	E2_spurious_opener	layer 0, head 3	115	0.0358	0.0410

Table 13: Examples of direct attention to corrupted positions.

The attention mass around corrupted positions is small. The model does not consistently route strong attention toward the error token, nor does it show a clear head specialised for localising corrupted positions. This agrees with the correction results: the model does not reliably localise errors, especially in OOD strings.

14 Global [CLS] depth probe

The required global probe asks whether the final-layer [CLS] representation linearly encodes maximum nesting depth. Two probes were trained on frozen [CLS] representations:

- a Ridge regression probe predicting maximum depth, evaluated with R^2 and MAE;
- a logistic-regression classifier predicting discrete depth, evaluated with accuracy and macro-F1.

Probe	Split	Rows	R^2	Accuracy	Macro-F1
Regression	probe_test	700	0.554	–	–
Classification	probe_test	700	–	34.86%	33.02%
Regression	dev	1000	-0.367	–	–
Classification	dev	1000	–	30.90%	18.24%
Regression	test_id	1000	-0.345	–	–
Classification	test_id	1000	–	31.50%	18.92%
Regression	test_ood	1000	-0.785	–	–
Classification	test_ood	1000	–	35.40%	20.34%

Table 14: Global depth probing from the final-layer [CLS] representation.

The [CLS] vector contains some linearly recoverable depth signal on the balanced synthetic probe test set. However, the negative R^2 values on dev, test_id, and test_ood show that this signal does not transfer cleanly to the actual evaluation distributions. Maximum depth is not robustly encoded in [CLS] in a simple linear form.

15 Local depth probe by layer

The local probe predicts the current nesting depth after each token from frozen token representations. This was done layer by layer and evaluated on both valid and corrupted strings.

For corrupted strings, the results are split into positions before, at, and after the corrupted token.

In this table, layers 1 and 2 correspond to Transformer blocks indexed as 0 and 1 in the implementation.

Layer	Split / position	R^2	MAE
1	dev valid	0.286	0.900
1	test_id valid	0.285	0.901
1	test_id corrupt, before error	0.319	0.856
1	test_id corrupt, at error	0.245	0.957
1	test_id corrupt, after error	-0.257	1.240
1	test_ood valid	-0.953	1.895
1	test_ood corrupt, all	-1.033	2.173
2	dev valid	0.277	0.905
2	test_id valid	0.278	0.905
2	test_id corrupt, before error	0.281	0.892
2	test_id corrupt, at error	0.197	1.017
2	test_id corrupt, after error	-0.206	1.217
2	test_ood valid	-0.530	1.743
2	test_ood corrupt, all	-0.769	2.087

Table 15: Layer-wise local depth probing.

Since the model has only two Transformer layers, the layer-wise trend is reported in table form rather than as a separate plot; the same R^2 -by-layer information is shown explicitly. Local depth is only partially decodable in-distribution and fails to extrapolate to OOD depths.

Local depth is partially decodable in-distribution, with R^2 around 0.28. On corrupted in-distribution strings, the probe works better before the error than after it. After the corrupted position, R^2 becomes negative. On OOD strings, R^2 is negative for both valid and corrupted examples.

The probe predictions are therefore not sufficient to localise the error without direct supervision. They show that representations become less depth-consistent after corrupted positions, but this degradation is diffuse: it indicates that something went wrong, not exactly where it went wrong.

This suggests that token representations encode partial shallow-depth information, but not a stable stack variable that extrapolates to deeper nesting.

16 Additional token-level stack probes

An earlier probe also predicted two token-level stack variables on valid strings:

- `depth_after`: stack depth after reading each token;
- `top_closer_after`: expected next closer at the top of the stack.

Task	Split	Rows	Accuracy	Macro-F1
depth_after	dev	48284	63.01%	56.38%
depth_after	test_id	48568	63.77%	57.29%
depth_after	test_ood	117318	35.02%	21.89%
top_closer_after	dev	48284	80.21%	79.54%
top_closer_after	test_id	48568	80.74%	80.18%
top_closer_after	test_ood	117318	70.94%	66.72%

Table 16: Token-level stack probes on valid strings.

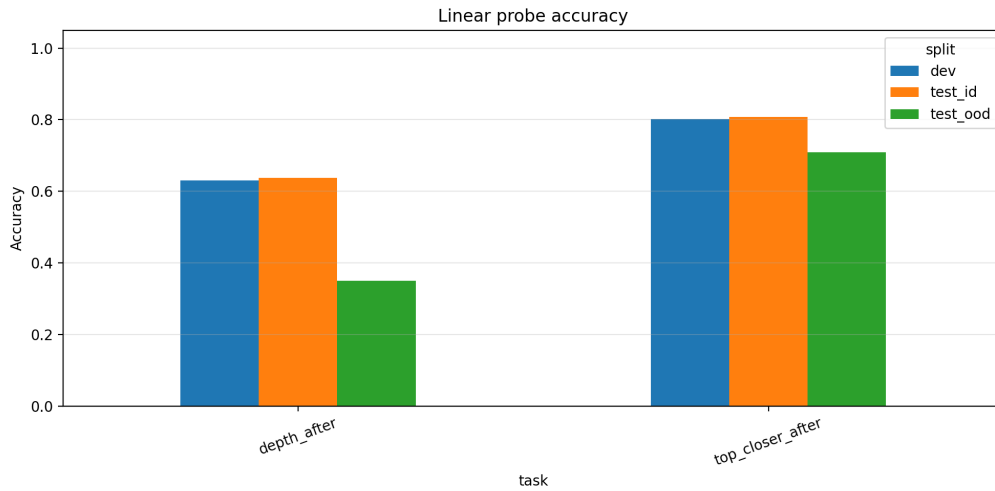


Figure 6: Linear probe accuracy for token-level stack variables.

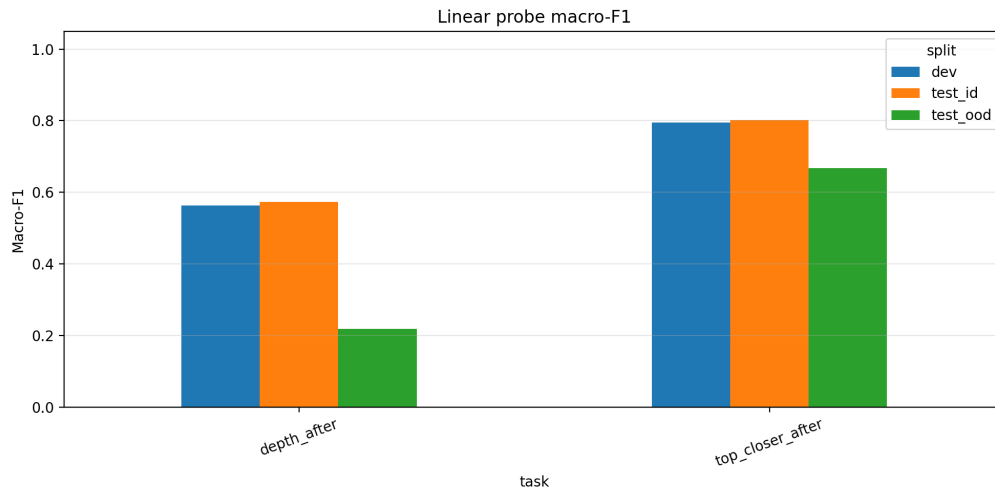


Figure 7: Linear probe macro-F1 for token-level stack variables.

The top-closer probe performs better than the depth probe, which means that some stack-top information is present in token representations. However, this information is not strong enough to make the detector robust. The model still fails badly on E3 type mismatches and on deeper OOD strings.

17 Rule learning or distribution-specific learning?

The evidence points to distribution-specific learning rather than full rule learning.

The model is not merely memorising training examples: it generalises from training to development and `test_id`. But it does not generalise to deeper strings. The PDA baseline gets 100% across all splits, while the Transformer collapses on depths 5–7.

The E3 weakness is especially important. Type mismatch detection requires comparing a closer against the most recent unmatched opener. This is the core typed-stack operation. The model handles easier corruption types much better than E3, which suggests that it relies on imbalance, local cues, and shallow distributional patterns.

The probing results support the same conclusion. Some stack information is decodable, especially in token representations, but it is partial and shallow. It does not extrapolate reliably to OOD depth.

18 Relevance to LLM syntactic abilities

This lab is relevant to LLMs because natural language and code both contain hierarchical dependencies:

```
nested clauses
agreement dependencies
scope relations
parentheses
programming syntax
HTML/XML tags
```

Dyck languages isolate one simple form of hierarchy: matching an opener with the correct closer. The Transformer learns useful structural cues in-distribution, but fails on deeper OOD nesting. This shows that good performance on familiar patterns is not enough evidence for abstract rule learning.

Two important disanalogies limit the conclusion. First, Dyck languages are purely formal and deterministic, while natural language syntax interacts with meaning, ambiguity, frequency, and discourse context. Second, Dyck strings isolate one kind of nested dependency, whereas natural language contains overlapping dependencies, optional constituents, agreement phenomena, and semantic constraints.

Therefore, failure on Dyck OOD generalisation does not directly imply that LLMs cannot parse natural language. But it does show that in-distribution success is not enough evidence for abstract structural generalisation.

19 Final conclusion

The Transformer learns useful in-distribution heuristics for Dyck error detection. It reaches about 87.6% accuracy on `test_id`, but only 38.7% on `test_ood`. Fine-tuning on 500 depth-5 examples raises raw OOD accuracy to about 50%, but macro-F1 remains low. Curriculum learning improves OOD performance to about 48–49%, but still remains far below in-distribution accuracy.

For correction, the repaired-string evaluation shows that the model rarely fixes corrupted strings: only 13.64% of corrupted `test_id` strings and 5.64% of corrupted `test_ood` strings are repaired into valid Dyck strings. This is much lower than token accuracy and confirms that correction remains a hard localisation problem.

The interpretability results are consistent with this. Attention heads show weak matching-bracket sensitivity, but not a robust stack mechanism. The global `[CLS]` probe does not robustly encode maximum depth across evaluation splits. Local token probes show partial shallow stack-state information, especially before corrupted positions, but fail on deeper OOD strings.

Overall, the model learns useful patterns over the training distribution, but not the full typed pushdown-stack algorithm for $D(2)$.

Data files used

The numerical results in this report come from:

- `data/dataset_summary.csv`
- `data/probe_depth_train.csv`
- `data/probe_depth_test.csv`
- `data/finetune_depth5.csv`
- `outputs/tables/detection_metrics.csv`
- `outputs/tables/eval_overall_metrics.csv`
- `outputs/tables/eval_by_depth.csv`
- `outputs/tables/eval_by_error_type.csv`
- `outputs/tables/eval_by_length_bin.csv`
- `outputs/tables/correction_metrics.csv`
- `outputs/tables/correction_repair_metrics.csv`
- `outputs/tables/correction_repair_by_error_type.csv`
- `outputs/tables/correction_predictions_test_id_repair.csv`
- `outputs/tables/correction_predictions_test_ood_repair.csv`

- outputs/tables/pda_baseline_metrics.csv
- outputs/tables/finetune_depth5_by_depth.csv
- outputs/tables/curriculum_detection_by_depth.csv
- outputs/tables/global_cls_depth_probe_metrics.csv
- outputs/tables/local_depth_probe_layer_metrics.csv
- outputs/tables/probe_metrics.csv
- outputs/attention/attention_head_summary_with_std.csv
- outputs/attention/error_attention_summary_test_id.csv
- outputs/attention/error_attention_summary_test_ood.csv
- outputs/figures/training_curves.png
- outputs/figures/detection_accuracy_by_depth.png
- outputs/figures/detection_accuracy_by_error_type.png
- outputs/figures/detection_accuracy_by_length.png
- outputs/figures/detection_accuracy_by_split.png
- outputs/figures/detection_macro_f1_by_split.png
- outputs/figures/attention_matching_opener.png
- outputs/figures/probe_accuracy.png
- outputs/figures/probe_macro_f1.png